You define all these options when creating a CORS policy in your call to `AddCors()` using the `CorsPolicyBuilder`, as you saw in listing 29.2. A policy can set all or none of these options, so you can customize the results to your heart's content. Table 29.1 shows some of the options available and their effects.

Table 29.1 The methods available for configuring a CORS policy and their effect on the policy

`CorsPolicyBuilder` method example	Result
`WithOrigins("http://shopping.com")`	Allows cross-origin requests from http://shopping.com
`AllowAnyOrigin()`	Allows cross-origin requests from any origin. This means any website can make JavaScript requests to your API.
`WithMethods()`/`AllowAnyMethod()`	Sets the allowed methods (such as `GET`, `POST`, and `DELETE`) that can be made to your API
`WithHeaders()`/`AllowAnyHeader()`	Sets the headers that the browser may send to your API. If you restrict the headers, you must include at least `Accept`, `Content-Type`, and `Origin` to allow valid requests.
`WithExposedHeaders()`	Allows your API to send extra headers to the browser. By default, only the `Cache-Control`, `Content-Language`, `Content-Type`, `Expires`, `Last-Modified`, and `Pragma` headers are sent in the response.
`AllowCredentials()`	By default, the browser won't send authentication details with cross-origin requests unless you explicitly allow it. You must also enable sending credentials client-side in JavaScript when making the request.

One of the first problems in setting up CORS is realizing you have a cross-origin problem at all. Several times I've been stumped trying to figure out